



NEXUS TECH RECRUITMENT

2026

TECHNICAL DEPARTMENT • FRESHERS RECRUITMENT

**BUILD SOMETHING
THAT MAKES STUDENT LIFE EASIER.**

A beginner-friendly, project-based recruitment challenge. AI and vibecoding are encouraged — understanding what you build matters more than how much code you write.



01 • CHALLENGE OVERVIEW

The idea: Build a small application or tool that solves one real problem faced by students. This challenge is intentionally designed for students who have only recently started college. You are not expected to know advanced frameworks or build production-scale software.

WHAT NEXUS IS LOOKING FOR

Curiosity • logical thinking • willingness to learn • ability to turn an idea into a working prototype • good use of modern tools • honest understanding of your own work.

Choose ONE problem statement

Pick one of the following challenges. You may implement the suggested features and add your own ideas. Bonus ideas are optional and are meant to help stronger candidates differentiate themselves.

#	PROBLEM	SUMMARY
01	AI STUDY BUDDY	Learn difficult concepts through an AI-powered study companion.
02	CAMPUS LOST & FOUND	Help students report, discover and recover lost belongings.
03	SMART EVENT DISCOVERY	Help students discover campus events relevant to them.
04	AI RESUME ANALYZER	Help students understand and improve their resumes.
05	COLLEGE SURVIVAL ASSISTANT	Make common campus information easier to find.

AI + Vibecoding policy

AI-assisted development is encouraged. You may use ChatGPT, Gemini, Claude, Cursor, GitHub Copilot, v0, Lovable, Bolt or similar tools for ideation, learning, UI generation, coding, debugging, documentation and testing. You must still be able to explain the final product.

IMPORTANT

Using AI will not reduce your score. Hiding or misrepresenting AI usage is worse than using AI heavily and explaining it well.



01 — AI STUDY BUDDY

Problem: Students often struggle to understand difficult concepts, revise before exams and find personalized learning material. Build an AI-powered study companion that makes learning more interactive.

Minimum expected functionality:

- A student enters a topic or concept.
- The system provides a simple explanation, examples and/or real-world applications.
- The system can generate practice questions or a short quiz.
- The student can receive a concise revision summary.

Optional / AI enhancement ideas:

Upload notes or PDFs and ask questions about them; generate flashcards; adjust explanation difficulty; create a study plan; track weak topics.

What we evaluate:

How well you understand user problems and integrate AI meaningfully.



02 — CAMPUS LOST & FOUND PLATFORM

Problem: Students lose items around campus and may not know where to report them or how to find the person who discovered them. Build a simple platform that connects lost-item reports with found-item reports.

Minimum expected functionality:

- Post a lost item with name, description, location and date.
- Post a found item with similar details.
- Search and filter posts by category, location or date.
- Provide a safe way for a finder and owner to connect.

Optional / AI enhancement ideas:

Image similarity, automatic category detection, smart matching, suggested possible matches and duplicate detection.

What we evaluate:

Database design, user experience thinking and practical problem solving.



03 — SMART EVENT DISCOVERY SYSTEM

Problem: VIT has many events, workshops and activities. Students can easily miss opportunities because information is spread across different channels. Build a simple system for discovering and managing events.

Minimum expected functionality:

- Display upcoming events with useful details.
- Allow students to search or filter events.
- Allow registration or a clear registration action.
- Allow students to save/favourite events.

Optional / AI enhancement ideas:

Personalized event recommendations, AI-generated event summaries, natural-language search and an AI event assistant.

What we evaluate:

Product thinking, UI design and ability to create useful systems.



04 — AI RESUME ANALYZER

Problem: Freshers often do not know whether their resume communicates their skills clearly or what recruiters expect. Build an AI tool that gives constructive, understandable feedback.

Minimum expected functionality:

- Accept a resume as text or an uploaded document, where technically feasible.
- Analyze important sections such as education, projects and skills.
- Give clear improvement suggestions.
- Highlight missing, weak or unclear information.

Optional / AI enhancement ideas:

ATS-style analysis, job-description matching, skill-gap suggestions, project-description rewriting and a personalized improvement checklist.

What we evaluate:

AI usage, information processing and user-focused design.



05 — COLLEGE SURVIVAL ASSISTANT

Problem: Freshers have countless questions about campus life, facilities, clubs, procedures and everyday college navigation. Build an assistant that makes useful information easier to access.

Minimum expected functionality:

- Organize frequently asked campus questions and answers.
- Allow students to search for information.
- Present answers in a clear and beginner-friendly format.
- Clearly indicate when the system does not know an answer.

Optional / AI enhancement ideas:

AI chatbot, college knowledge base, retrieval-augmented answers, source references and context-aware follow-up questions.

What we evaluate:

Ability to build helpful systems and apply AI responsibly.

You may go beyond the brief

The listed features are a starting point, not a ceiling. You may add features, change the UI, use a different technology stack or approach the problem differently. Do not add complexity just to appear advanced. A small feature that is well understood is better than five copied features.



02 • MANDATORY SUBMISSION REPORT

SHORTLISTING REQUIREMENT

The project report must be submitted before the interview. The report will be used as part of the shortlisting process. A project without the required report may not be considered for the interview round.

Report format

Keep the report concise and readable. A suggested length is 4–8 pages, excluding screenshots if necessary. Use the following structure:

1. Candidate Details	Name, VIT registration number, branch/programme, year/section and contact information as required by the recruitment form.
2. Project Details	Project title, selected problem statement, one-paragraph project summary and the main problem you decided to solve.
3. Problem Understanding	Who is the user? What problem are they facing? Why is it worth solving? What assumptions did you make?
4. Solution & Features	Explain your solution, minimum features implemented and any additional features. Include screenshots where useful.
5. Tech Stack	Frontend, backend, database, APIs, AI model/provider, libraries/frameworks and hosting/deployment technology.
6. Architecture / Flow	A simple architecture diagram or flow diagram showing how the major parts of your application interact.
7. Implementation	Briefly explain important technical decisions. Do not paste the entire source code into the report.
8. Challenges & Debugging	Describe at least one meaningful problem/bug you faced, what caused it, how you investigated it and how you fixed it.
9. Learning & Reflection	What did you learn? What would you improve with more time? What part of the project do you understand best?
10. Links	GitHub repository link — mandatory. Deployment/live demo link — include if deployed. Add any other relevant links only if useful.



03 • AI / VIBECODING REPORT

We actively encourage AI-assisted development. The purpose of this section is not to catch you using AI — it is to understand whether you can direct, verify, debug and learn from AI.

A. Tools used

- List every significant AI tool used. Example: ChatGPT, Gemini, Claude, Cursor, GitHub Copilot, v0, Lovable, Bolt.
- Explain what each tool was used for rather than simply listing names.

B. Your workflow

RECOMMENDED WORKFLOW

IDEA → RESEARCH → PLAN → AI ASSISTANCE → BUILD → TEST → DEBUG → REVIEW → DEPLOY

Explain what you personally did at each stage. Your workflow does not have to match this example.

C. Prompt evidence

Include 2–5 important prompts. For each prompt, explain:

- What you were trying to achieve.
- Why you wrote the prompt that way.
- What the AI produced.
- What you accepted, rejected or modified.

D. AI-generated code audit

Choose one meaningful piece of AI-assisted code and document:

- What AI generated.
- What you changed.
- Why you changed it.
- How you verified that the final code works.

E. Challenges faced



Be honest. Examples include hallucinated APIs, incorrect code, UI bugs, dependency conflicts, poor prompts, AI misunderstanding the requirements, repeated failed fixes or difficulty understanding generated code.

THE BEST ANSWER IS NOT “AI DID EVERYTHING.”

A strong report shows that you can use AI quickly while still taking responsibility for the final system.

F. Verification

State how you tested the final product: manual testing, test cases, browser/device checks, API testing, logs, unit tests, database checks or other methods you actually used.



04 • JUDGING RUBRIC

The rubric is designed for freshers. We will not reward unnecessary complexity. A simple project that is functional, well understood and thoughtfully built can outperform a technically flashy project that the candidate cannot explain.

CATEGORY	WEIGHT	WHAT WE LOOK FOR
Problem Solving	40%	Understanding the problem, logical approach, feature prioritization, practical decisions.
Technical Implementation	25%	Working functionality, sensible structure, code quality, basic engineering practices.
AI / Vibecoding Workflow	20%	Effective AI use, prompt quality, verification, debugging and understanding of generated output.
Creativity	15%	Originality, thoughtful UX, useful additions and quality of execution.

Shortlisting checklist

- Project is submitted in the required format.
- GitHub repository link is provided and accessible.
- Deployment/live demo link is included if the project is deployed.
- Report explains the actual work done by the candidate.
- AI usage is disclosed clearly.
- Candidate can explain their technical decisions and debugging process.

What can make you stand out?

- A polished but simple user experience.
- A thoughtful solution to the actual problem rather than feature overload.
- A clear GitHub README with setup instructions.
- Good commit history or meaningful documentation of development.
- Honest and specific AI workflow documentation.
- A feature you built because you noticed a real user need.



NEXUS SELECTION PHILOSOPHY

We are not selecting experienced developers. We are selecting future developers.

Don't show us what you know. Show us what you can build — and what you learned while building it.